# Trabajo práctico · Unidad 7 — NoSQL y nuevas formas de almacenamiento

Bases de Datos II · TUPaD UTN · Docente: Carlos Yácomo · **Grupo H (Saferazi)**: Lautaro Fernandez, Daniela Díaz, Jesús Ramírez, Danilo Serrano, Elio Marí
Repositorio: https://github.com/lauticodemaster/UTN-TUPaD-Base-de-datos-2 (carpeta `TP9 (Unidad 7)`) · Entorno: PostgreSQL 17 y MongoDB 8.0 (contenedores Docker)

## 1. Modelo documental (consigna 1)

**Subconjunto relacional.** En el Food Store del TP1 intervienen:

| Tabla | Clave primaria | Claves foráneas y restricciones relevantes |
|---|---|---|
| `pedido` | `id` | `usuario_id → usuario.id`; `estado` ENUM (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO); `total NUMERIC(10,2) ≥ 0`; `forma_pago` ENUM; `eliminado` |
| `detalle_pedido` | `id` | `pedido_id → pedido.id` (ON DELETE RESTRICT); `producto_id → producto.id`; `UNIQUE(pedido_id, producto_id)`; `cantidad > 0`; `precio_unitario` congelado por `trg_subtotal` |
| `usuario` | `id` | Datos del cliente: solo se copian nombre y apellido |
| `producto` / `categoria` | `id` | `producto.categoria_id → categoria.id`: se copian nombre de producto y de categoría en cada línea |

**Colección propuesta:** `foodstore.pedidos`, un documento por pedido.

```
{
  _id: 4,              // = pedido.id
  fecha: ISODate("2026-10-14"),
  estado: "CONFIRMADO",
  forma_pago: "TARJETA",
  total: Decimal128("3060.00"),
  cliente: {           // objeto embebido
    usuario_id: 2,     // = usuario.id
    nombre: "Juliana", apellido: "Paredes" },
  lineas: [            // arreglo embebido
    { detalle_id: 9,   // = detalle_pedido.id
      producto_id: 3,  // = producto.id
      nombre: "Jamón y Queso", categoria: "Pizzas",
      cantidad: 1,
      precio_unitario: Decimal128("2000.00"),
      subtotal: Decimal128("2000.00") }, ... ],
  origen: "prueba TP Unidad 7"
}
```

- **Identificadores conservados:** `_id = pedido.id`, `cliente.usuario_id`, `lineas.detalle_id` y `lineas.producto_id`. Así se puede volver a cada fila de PostgreSQL y comparar resultados por id.
- **Embebido:** `cliente` (objeto) y `lineas` (arreglo). Las líneas no existen sin su pedido y se leen siempre con él.
- **Snapshot histórico:** el nombre del producto, la categoría y el `precio_unitario` se copian tal como estaban al comprar. Si después cambia el catálogo, el pedido no se modifica.
- **Importes en `Decimal128`**, equivalente a `NUMERIC(10,2)`, para no perder centavos con `double`.
- **Sin datos sensibles:** no se migran mail, celular ni contraseña.
- **Índices:** `{estado: 1}` y el multikey `{"lineas.producto_id": 1}`.

## 2. Migración (consigna 2)

**Procedencia de los datos.** Pedidos 1 a 3: base de desarrollo del equipo (`TP1/data.sql`, con fechas y totales del backup). Pedidos 4 a 7: **datos de prueba** cargados con `sql/datos_prueba_postgresql.sql` mediante el procedimiento `sp_crear_pedido` del propio Food Store. Ese procedimiento valida el stock y los triggers calculan subtotales y totales, así que ningún importe se escribió a mano. Total: **7 pedidos y 19 líneas**.

**Proceso.** `sql/exportar_pedidos_mongodb.sql` genera desde PostgreSQL el literal de cada documento (con `format()` y `string_agg`). Ese texto se pegó en el `insertMany()` de `consultas_mongodb.js`, que antes crea la colección con un validador (`$jsonSchema` + `$expr`).

```
$ mongosh --quiet --file consultas_mongodb.js
Documentos insertados: 7
== Verificación: documentos con importes incoherentes (esperado: ninguno) ==
[]
```

```
== Verificación: el validador rechaza una línea con cantidad 0 ==
Rechazado: Document failed validation
== Verificación: el validador rechaza un total que no coincide con sus líneas ==
Rechazado: Document failed validation
```

## 3. Reglas de integridad y criterio de embebido (consigna 3)

- **Regla 1 · Coherencia de importes:** `subtotal = cantidad × precio_unitario` en cada línea y `total = Σ subtotales` del pedido. En PostgreSQL la garantizan `trg_subtotal` y `trg_total_*`. En MongoDB se migran los valores calculados en origen, en `Decimal128`, y la regla se hace cumplir con un validador `$expr` (`$sum`, `$multiply`, `$allElementsTrue`). Los 7 documentos lo cumplieron, una agregación de control lo confirmó (resultado: `[]`) y se rechazó un documento de prueba con total $5000 y líneas que sumaban $3600.
- **Regla 2 · Dominios y valores válidos:** el `estado` y la `forma_pago` solo admiten los valores de los ENUM, `cantidad` es un entero ≥ 1, los importes son ≥ 0 y cada pedido tiene al menos una línea. Esto replica los `CHECK`/ENUM de PostgreSQL con el validador `$jsonSchema`, que rechazó un documento con `cantidad: 0`. Además se conservan los ids de origen y se migran solo filas con `eliminado = FALSE`.

**Qué se embebe y qué no.** Se embeben las líneas y un resumen del cliente porque el acceso frecuente es «pedido completo» y esos datos tienen relación *contiene* con el pedido: se leen y escriben juntos, y su cantidad está acotada (1 a 4 líneas). Por eso una sola lectura reemplaza los `JOIN` de `pedido`, `detalle_pedido`, `producto` y `usuario`. **No** se embeben el catálogo completo (precio vigente, stock, disponibilidad) ni la cuenta del usuario: cambian de forma independiente, se comparten entre muchos pedidos y duplicarlos obligaría a actualizar todos los documentos. Del producto solo se guarda el snapshot de la venta.

## 4. Consultas en MongoDB (consigna 4)

| Consulta | Instrucción | Resultado (ids) |
|---|---|---|
| a) Todos los pedidos (id y estado) | `db.pedidos.find({}, {_id: 1, estado: 1}).sort({_id: 1})` | 1 CONFIRMADO · 2 PENDIENTE · 3 TERMINADO · 4 CONFIRMADO · 5 TERMINADO · 6 CANCELADO · 7 TERMINADO |
| b) Estado TERMINADO | `db.pedidos.find({estado: "TERMINADO"}, {_id:1, estado:1, fecha:1, total:1})` | 3, 5, 7 (3 pedidos) |
| c) Contiene el producto 7 (Gaseosa 2L) | `db.pedidos.find({"lineas.producto_id": 7}, {_id:1, estado:1, "lineas.$":1})` | 1, 4, 5, 7 (4 pedidos) |
| d) Total > $2500 | `db.pedidos.find({total: {$gt: Decimal128("2500")}}, {_id:1, estado:1, total:1})` | 1 ($3950), 4 ($3060), 7 ($3750) |

**Por qué la consulta c) usa el id.** Filtra por `producto_id` y no por nombre, porque el id es estable y se puede comparar con PostgreSQL. La proyección `"lineas.$"` devuelve solo la línea que coincide. Según `explain("executionStats")`, usa `IXSCAN` sobre `lineas.producto_id_1` y examina 4 documentos para devolver 4.

**Umbral de la consulta d).** El promedio de los 7 totales es 17210 / 7 = **$2458,57** (MongoDB devolvió `2458.5714…` con `$avg`), y se redondeó a $2500 para obtener los pedidos por encima de la media. Además, el pedido 3 vale exactamente $2500 y quedó excluido. Eso confirma que `$gt` es estricto y que la comparación en `Decimal128` es exacta.

```
== b) Pedidos con estado TERMINADO ==
[ { _id: 3, fecha: ISODate('2026-09-01T00:00:00.000Z'), estado: 'TERMINADO', total: Decimal128('2500.00')
},
  { _id: 5, fecha: ISODate('2026-10-21T00:00:00.000Z'), estado: 'TERMINADO', total: Decimal128('1770.00')
},
  { _id: 7, fecha: ISODate('2026-11-02T00:00:00.000Z'), estado: 'TERMINADO', total: Decimal128('3750.00') }
]
Cantidad: 3
== c) Pedidos que contienen el producto 7 (Gaseosa 2L) ==   (extracto)
[ { _id: 1, estado: 'CONFIRMADO', lineas: [ { detalle_id: 2, producto_id: 7, nombre: 'Gaseosa 2L',
cantidad: 1, ... } ] },
  { _id: 4, estado: 'CONFIRMADO', lineas: [ { detalle_id: 10, producto_id: 7, cantidad: 2, subtotal:
```

```
Decimal128('700.00') } ] }, ...
Cantidad: 4
Etapa: IXSCAN | índice: lineas.producto_id_1 | docs examinados: 4 | devueltos: 4
```

La salida completa está en `evidencias/salida_mongodb.txt`.

## 5. Verificación con PostgreSQL (consigna 5)

```
-- b) Estado TERMINADO
SELECT p.id, p.fecha, p.estado, p.total
FROM   pedido p
WHERE  p.estado = 'TERMINADO'
  AND  p.eliminado = FALSE
ORDER  BY p.id;
```

```
-- c) Contiene el producto 7
SELECT p.id FROM pedido p
WHERE  p.eliminado = FALSE
  AND  EXISTS (SELECT 1 FROM detalle_pedido d
       WHERE d.pedido_id = p.id
         AND d.producto_id = 7
         AND d.eliminado = FALSE);
```

| Consulta | Cantidad en PostgreSQL | Cantidad en MongoDB | IDs | Coinciden los IDs |
|---|---|---|---|---|
| Estado seleccionado (TERMINADO) | 3 | 3 | 3, 5, 7 | Sí |
| Producto seleccionado (id 7, Gaseosa 2L) | 4 | 4 | 1, 4, 5, 7 | Sí |

La consulta d) también coincide (1, 4, 7), igual que la cantidad de líneas por pedido (2, 3, 3, 3, 3, 2, 3) y los 7 totales. Salida completa en `evidencias/salida_postgresql.txt`.

**Diferencias detectadas y correcciones:**

- **Borrado lógico.** En PostgreSQL las filas «borradas» siguen en la tabla con `eliminado = TRUE`, y la migración solo lleva las vigentes. Si las consultas SQL no filtran `eliminado = FALSE` en *ambas* tablas, los conjuntos pueden diferir. **Corrección:** se agregó el filtro en todas las consultas. Además, en la consulta c) se usa `EXISTS` en lugar de `JOIN`, porque `find()` devuelve documentos y no líneas.
- **Desincronización después de migrar** (`sql/experimento_divergencia.sql`, ejecutado con `ROLLBACK`). Se dio de baja la línea de Gaseosa del pedido 5. En PostgreSQL, el trigger recalculó el total (1770 → **1420**) y la consulta c) pasó a devolver **3** pedidos (1, 4, 7). MongoDB siguió devolviendo **4** (1, 4, 5, 7) con total 1770. La copia documental es una foto: si convive con PostgreSQL, necesita un mecanismo de sincronización (por ejemplo, *change data capture*) o volver a migrar.
- **Tipos numéricos.** Se verificó en el laboratorio que `mongosh` guarda los enteros literales como `int`, por lo que el validador puede exigir `bsonType: "int"` en ids y cantidades. Los importes se fuerzan a `Decimal128`: un literal como `3950.00` se guardaría como `double`.
- **Fechas.** `DATE` de PostgreSQL pasa a `ISODate` a las 00:00 UTC. Si un cliente lo muestra en hora argentina (UTC−3), lo ve como el día anterior a las 21:00. Hay que filtrar por rango UTC o guardar la fecha local como texto ISO.

## 6. Evaluación con IA (consigna 6)

El prompt usado está en `ia/prompt_consigna6.md`. Menciona el modelo relacional actual (tablas, FK, triggers y procedimiento transaccional), el acceso frecuente a pedidos completos y la necesidad de mantener coherentes las cantidades, los precios históricos y el stock.

**Herramienta:** Claude (modelo Claude Opus 5.5, Anthropic), consultado desde Claude Code el 08/11/2026. La respuesta completa está en `ia/respuesta_ia.md`. **Resumen:** la IA propone un documento por pedido con las líneas embebidas y el snapshot de precios. Destaca que leer el pedido completo es una sola operación, pero que los reportes que cruzan entidades son más simples en SQL. Señala que la escritura de un documento es atómica y que descontar stock en otra colección exige transacciones multidocumento. Recomienda `Decimal128` para importes y concluye que conviene mantener PostgreSQL como fuente de verdad, con MongoDB solo como almacén de lectura sincronizado.

**Advertencia metodológica:** la misma herramienta armó el laboratorio, así que su respuesta no es independiente de él. Por eso las dos afirmaciones se contrastan con la documentación oficial y con pruebas ejecutadas (`laboratorio_ia.js`), no con la opinión de la IA.

## 7. Contraste de afirmaciones y conclusión (consigna 7)

| Afirmación de la IA | Contraste (documentación oficial y laboratorio) | Veredicto |
|---|---|---|
| **A1.** «Una operación de escritura sobre un único documento es atómica, incluso si modifica documentos embebidos». Para descontar stock en otra colección hacen falta transacciones multidocumento (replica set). | **Documentación:** el manual («Atomicity and Transactions») indica que las escrituras son atómicas a nivel de documento único y que, para atomicidad entre varios documentos, MongoDB ofrece transacciones en replica sets y clústeres fragmentados. **Laboratorio:** un `updateOne` cambió una línea embebida y el total en una sola operación. Un update que cambiaba la línea sin el total se rechazó entero, sin cambios parciales. Una transacción pedido + stock falló en el contenedor standalone («does not support retryable writes»), sin escribir nada. En un replica set de un nodo se confirmó: se creó el pedido 8 y el stock bajó de 30 a 28. | **Confirmada, con matiz:** que la escritura sea atómica no garantiza que el documento quede coherente. El update incoherente lo rechazó el validador `$expr`, no la atomicidad. |
| **A2.** Los importes no deben guardarse como `double`, porque no representa exactamente fracciones decimales. `Decimal128` es el equivalente de `NUMERIC`. | **Documentación:** «Model Monetary Data» afirma que el punto flotante binario (`double`) no puede representar fracciones decimales exactas, y que `decimal128` no aproxima valores decimales y da la precisión exacta que requieren los datos financieros. **Laboratorio** (aritmética en el servidor, `$add`/`$sum`): con `double`, 0,10 + 0,20 = `0.30000000000000004` (≠ 0,30) y 3 × 1800,10 = `5400.299999999999`. Con `Decimal128` se obtiene `0.30` y `5400.30`, y la suma de los 7 totales migrados da exactamente `17210.00`, igual que en PostgreSQL. | **Confirmada.** Justifica haber migrado los importes como `Decimal128`. |

### Conclusión

Conviene **mantener PostgreSQL** como sistema de registro de los pedidos de Food Store y no migrar este subconjunto a MongoDB.

**Criterio 1 · Integridad transaccional.** Crear un pedido no es solo escribir un documento: `sp_crear_pedido` bloquea el producto con `FOR UPDATE`, descuenta stock y registra líneas y total en una sola transacción, y los triggers garantizan `subtotal = cantidad × precio` y `total = Σ subtotales`. En MongoDB, el validador aseguró la coherencia dentro de cada documento, pero no puede controlar el stock ni la existencia del producto o del usuario en otras colecciones: eso queda a cargo de la aplicación y de transacciones multidocumento.

**Criterio 2 · Consistencia entre copias.** El experimento mostró que, al modificar un pedido en PostgreSQL, MongoDB conserva datos viejos (total 1770 frente a 1420). Combinar motores obliga a sincronizarlos.

**Criterio 3 · Rendimiento de lectura.** El documento embebido sí resuelve el «pedido completo» en una lectura indexada, sin JOIN. Pero PostgreSQL ya resuelve esa lectura con índices y la vista `v_pedido_detalle`.

MongoDB tendría sentido solo como réplica de lectura alimentada por eventos, si el historial de pedidos creciera mucho.